

Мессенджер

Проектный отчёт по практике — кейс №1

Сервер и клиент для быстрого обмена сообщениями с авторизацией, чатами и поиском.

Стек: Go, gRPC, PostgreSQL, Redis Streams, Bubble Tea (TUI)

Команда: [ФИО участников]

VK Education · 2026

Содержание

1. Предметная область и постановка задачи
2. Анализ требований
3. Архитектура решения
4. Путь сообщения и гарантии надёжности
5. Структура базы данных (ER-модель)
6. API сервиса
7. Роли и права доступа
8. Модель авторизации
9. SQL-запросы
10. Производительность
11. Соответствие требованиям и артефактам защиты
12. План работ и распределение вклада
13. Выводы
14. Источники

1. Предметная область и постановка задачи

Мессенджер — сервис обмена текстовыми сообщениями в реальном времени. Ключевые сущности предметной области: пользователь, чат (личный 1-на-1 и групповой), сообщение, участие пользователя в чате с определённой ролью. Особенность класса систем — push-доставка от сервера к клиенту с задержкой, незаметной для человека (< 200 мс), и гарантии надёжности: сообщение не должно теряться, порядок в рамках одного диалога должен сохраняться.

Цель работы — спроектировать и реализовать MVP мессенджера: сервер и клиент, авторизация, чаты, отправка/получение сообщений в реальном времени, история с пагинацией и поиск по сообщениям, — а также обосновать принятые архитектурные решения.

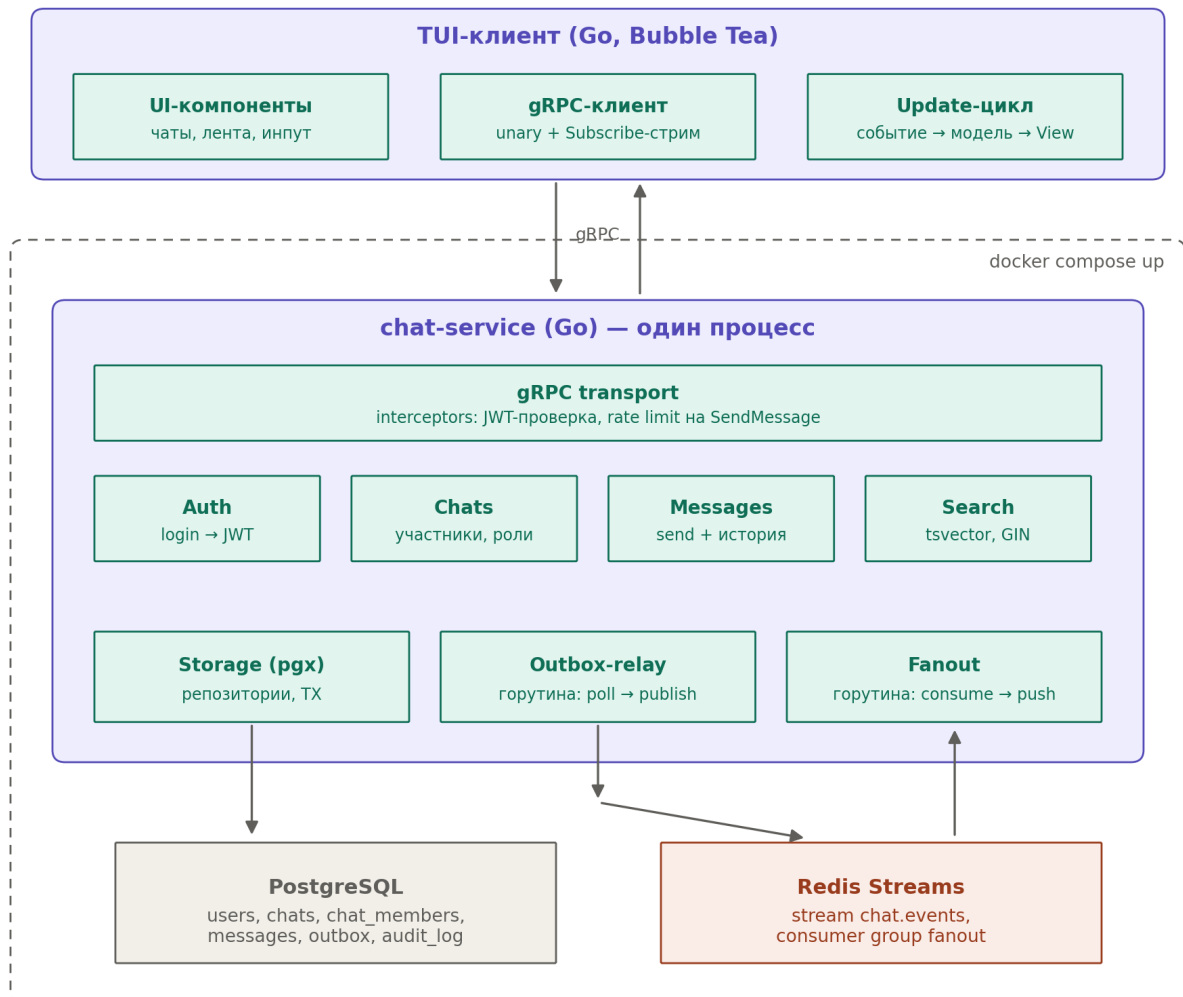
2. Анализ требований

Требования собраны из вводной встречи и воркшопа по кейсу №1. Для каждого зафиксировано проектное решение, которым оно закрывается.

№	Требование	Проектное решение
1	Регистрация/логин, JWT-авторизация	Register, Login → JWT; bcrypt-хэш пароля; auth-interceptor на всех методах, кроме Register/Login
2	Личный чат 1-на-1	chats.type = 'direct'; создаётся при первом сообщении
3	Групповой чат: создание, название	chats.type = 'group', поле title; создатель получает роль admin
4	Реалтайм-доставка (WebSocket/gRPC/long polling)	gRPC server-streaming Subscribe; конвейер outbox → Redis Streams → fanout
5	История сообщений, пагинация «вверх»	Keyset-пагинация по (chat_id, id DESC) с составным индексом; без OFFSET
6	Управление участниками — только администратором	Проверка role = 'admin' в модуле Chats; иначе gRPC-ошибка PermissionDenied
7	Поиск по сообщениям в рамках чата	Полнотекстовый поиск: tsvector + GIN-индекс (MVP-фолбек — ILIKE)
8	Безопасное хранение паролей	В БД только bcrypt-хэш; открытый пароль нигде не сохраняется и не логируется
9	Надёжность: недоставка исключена, порядок в диалоге	Transactional outbox; монотонный message_id в чате; ack после коммита
10	Ограничение частоты отправки (расширение)	Token bucket в gRPC-interceptor на SendMessage → ResourceExhausted
11	Статусы доставки/прочтения (расширение)	Read-купсор last_read_message_id в chat_members; событие message.read
12	UI (расширение)	Терминальный клиент (TUI) на Bubble Tea; нативный gRPC без гейтвея

3. Архитектура решения

Система сознательно спроектирована как один процесс chat-service с фоновыми горутинами вместо набора микросервисов: это проще эксплуатировать и объяснять, а надёжность обеспечивается паттерном transactional outbox, а не количеством сервисов. Клиент — терминальное приложение (TUI): оно использует нативный gRPC и снимает необходимость в отдельном гейтвее для браузера.



Компоненты. gRPC transport — единая точка входа, цепочка interceptor'ов (JWT-проверка, rate limit). Доменные модули Auth / Chats / Messages / Search — бизнес-логика, по одному пакету в internal/. Storage (pgx) — единственный слой с SQL; ключевой метод SaveMessage пишет messages и outbox в одной транзакции. Outbox-relay — горутина, публикующая неотправленные события в Redis Streams. Fanout — горутина, читающая поток событий и рассылающая их в открытые Subscribe-стримы участников чата.

Роль Redis Streams: (1) при горизонтальном масштабировании подписчик и отправитель могут быть подключены к разным экземплярам сервиса — поток событий их соединяет; (2) события переживают рестарт сервиса; consumer group даёт повторную доставку неподтверждённых событий.

Структура монорепо: api/proto (контракт), gen (генерация), cmd (chat-service, seed, tui), internal (auth, chat, storage, broker, search), migrations, deploy/docker-compose.yml, docs. Запуск: docker compose up.

4. Путь сообщения и гарантии надёжности

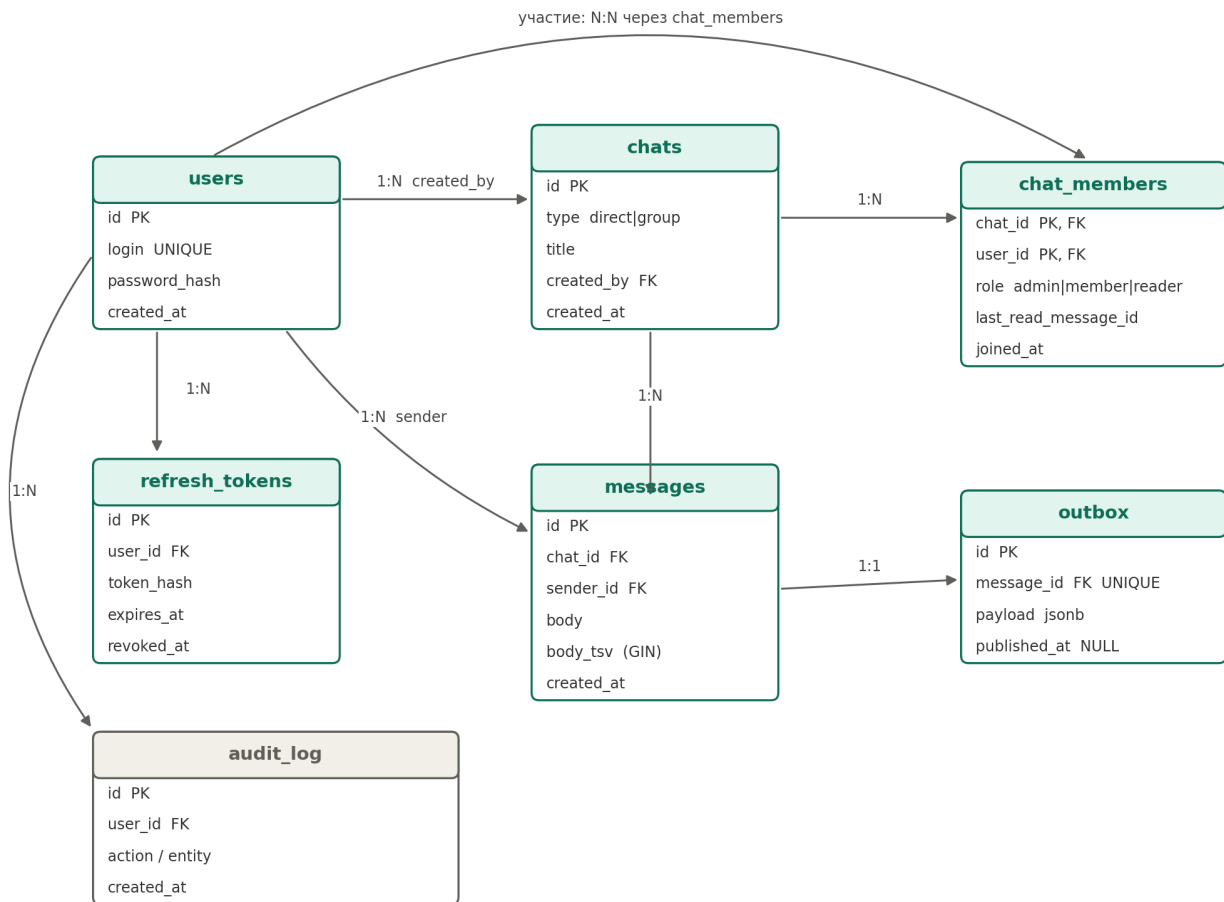
Отправка: инпут TUI → SendMessage → interceptor (JWT, rate limit) → Messages → Storage: транзакция INSERT message + INSERT outbox → ack отправителю (галочка «✓») → relay: XADD в chat.events → fanout: XREADGROUP → push во все Subscribe-стримы участников → Update-цикл получателя → перерисовка ленты.

Ключевые инженерные решения

Решение	Что даёт	Что происходит при сбое
Transactional outbox: сообщение и событие фиксируются одной транзакцией PG	«Принято» = «лежит в БД». Событие не может потеряться между БД и брокером	Redis недоступен → relay дойдёт после восстановления; published_at останется NULL до успеха
Аск отправителю сразу после коммита, доставка остальным — асинхронно	Мгновенный отклик UI (~5–10 мс), отправитель не ждёт получателей	Падение сервиса после коммита → сообщение уже сохранено, доставится при рестарте
Моноotonно растущий message_id в рамках чата	Порядок сообщений в диалоге; курсор для пагинации и read-статусов	Дубль события (at-least-once) → клиент дедуплицирует по message_id
Прочтение — такое же событие (message.read) в том же конвейере	Один механизм для new/read/typing; статусы «✓ / ✓✓» без отдельной инфраструктуры	Потеря события read не критична: курсор восстановится при следующем MarkRead

Статусы прочтения реализованы read-курсором: в chat_members хранится last_read_message_id. «Прочитано» = $id \leq$ курсора собеседника; в групповом чате «✓✓» — минимум курсоров участников. Это $O(1)$ записи на прочтение вместо строки на каждое сообщение × получателя.

5. Структура базы данных (ER-модель)



Модель содержит все типы связей: 1:1 (messages-outbox), 1:N (chats-messages, users-messages, users-refresh_tokens, users-audit_log), N:N (users-chats через связующую таблицу chat_members с составным первичным ключом).

Индексы

Индекс	Назначение
messages (chat_id, id DESC)	История чата: keyset-пагинация «вверх» без OFFSET
GIN (body_tsv)	Полнотекстовый поиск по сообщениям чата
outbox (id) WHERE published_at IS NULL — частичный	Дешёвый poll неотправленных событий relay'ем
chat_members (user_id)	Список чатов пользователя, проверка членства
users (login) UNIQUE	Логин и защита от дублей регистрации

6. API сервиса

Метод	Тип	Описание
Register(login, password)	unary	Регистрация; пароль хэшируется bcrypt
Login(login, password) → JWT	unary	Обмен учётных данных на токен доступа (+ refresh)

Метод	Тип	Описание
CreateChat(type, title, members)	unary	Создание личного или группового чата
GetChats() → [chat]	unary	Список чатов пользователя (только где он участник)
AddMember / RemoveMember	unary	Управление участниками; только для role = admin
SendMessage(chat_id, text) → message_id	unary	Отправка; транзакция message + outbox; ack после коммита
GetHistory(chat_id, before_id, limit)	unary	История с keyset-пагинацией «вверх»
Search(chat_id, query)	unary	Полнотекстовый поиск в рамках чата
MarkRead(chat_id, message_id)	unary	Сдвиг read-курсора; порождает событие message.read
Subscribe() → stream ServerEvent	server-streaming	Долгоживущий стрим событий: message.new, message.read, typing

Все методы, кроме Register и Login, требуют JWT в метаданных запроса; проверка выполняется общим interceptor'ом.

7. Роли и права доступа

Применён принцип минимальных привилегий; роль задаётся на пару (чат, пользователь) в chat_members.

Роль	Чтение	Отправка сообщений	Управление участниками	Управление ролями
reader (читатель)	Да	Нет	Нет	Нет
member (участник)	Да	Да	Нет	Нет
admin (администратор)	Да	Да	Да	Да

Изоляция данных: каждый запрос чтения (GetChats, GetHistory, Search) содержит JOIN chat_members ... AND user_id = \$current — пользователь физически не может получить сообщения чата, в котором не состоит.

8. Модель авторизации

Схема: клиент обменивает login + password на пару токенов — короткоживущий JWT (15 мин, подпись HMAC-SHA256, в payload: user_id, exp) и refresh-токен (7 дней, в БД хранится только его хэш, возможен отзыв через revoked_at). JWT передаётся в метаданных каждого gRPC-вызова; interceptor валидирует подпись и срок без запроса к БД.

Хранение секретов: пароли — только bcrypt-хэши (устойчив к перебору за счёт настраиваемой стоимости); refresh-токены — только хэши; JWT-секрет — переменная окружения, не в репозитории. Аудит: события login, send_message, add_member, remove_member, mark_read записываются в audit_log (кто, что, когда).

9. SQL-запросы

Демонстрационные запросы к схеме (JOIN, GROUP BY, подзапрос, оконная функция):

— Топ-5 чатов по числу сообщений за неделю (JOIN + GROUP BY):

```
SELECT c.title, COUNT(m.id) AS cnt
FROM chats c JOIN messages m ON m.chat_id = c.id
WHERE m.created_at > now() - interval '7 days'
GROUP BY c.id ORDER BY cnt DESC LIMIT 5;
```

— Число непрочитанных сообщений пользователя по чатам (подзапрос по read-курсуру):

```
SELECT cm.chat_id,
  (SELECT COUNT(*) FROM messages m
   WHERE m.chat_id = cm.chat_id AND m.id > cm.last_read_message_id) AS unread
FROM chat_members cm WHERE cm.user_id = $1;
```

— Последнее сообщение каждого чата для списка чатов (оконная функция):

```
SELECT * FROM (
  SELECT m.*, ROW_NUMBER() OVER (PARTITION BY chat_id ORDER BY id DESC) rn
  FROM messages m JOIN chat_members cm
  ON cm.chat_id = m.chat_id AND cm.user_id = $1
) t WHERE rn = 1;
```

— История с keyset-пагинацией и поиск:

```
SELECT id, sender_id, body, created_at FROM messages  
WHERE chat_id = $1 AND id < $2 ORDER BY id DESC LIMIT 50;
```

```
SELECT id, body FROM messages  
WHERE chat_id = $1 AND body_tsv @@ plainto_tsquery('russian', $2)  
ORDER BY id DESC LIMIT 20;
```

10. Производительность

Целевое требование — доставка «почти мгновенно» с точки зрения пользователя (< 200 мс). Раскладка задержки на пути сообщения (локальная сеть / docker):

Этап	Оценка задержки
gRPC-вызов клиент → сервер	0.5–2 мс
Транзакция INSERT message + outbox (PostgreSQL)	1–5 мс
Ack отправителю («✓» у отправителя)	итого 5–10 мс
Outbox-relay замечает запись (poll 50 мс или LISTEN/NOTIFY)	0–50 мс
XADD + XREADGROUP (Redis Streams)	1–2 мс
Push в стрим получателя + отрисовка TUI	1–5 мс
Итого конец-в-конец до экрана получателя	10–70 мс — запас ×3 от требования

Узкое место одно — интервал опроса outbox; устраняется связкой LISTEN/NOTIFY (pg_notify после коммита будит relay мгновенно, poll остаётся фолбеком). Меры, принятые против деградации: keyset-пагинация вместо OFFSET (глубокие страницы истории), GIN-индекс вместо ILIKE (поиск на больших объёмах — логарифмический), пул соединений pgxpool, частичный индекс на outbox. По пропускной способности запас на порядки: одиночный PostgreSQL держит тысячи INSERT/с, Redis Streams — десятки тысяч событий/с, Go — тысячи одновременных стримов на горутинах.

11. Соответствие требованиям и артефактам защиты

Артефакт / критерий	Чем закрывается	Статус
Отчёт по практике	Настоящий документ (все обязательные разделы, схемы)	черновик готов
ER-диаграмма	Раздел 5: все сущности, РК/FK, связи 1:1, 1:N, N:N	готово
Код сервисов	Монорепозиторий, ссылка на GitHub	в работе
Код скриптов	migrations/ + cmd/seed (наполнение демо-данными)	в работе
Модель авторизации	Раздел 8: JWT + refresh, bcrypt, хранение секретов	готово
Роли и права доступа	Раздел 7: 3 роли, минимальные привилегии, изоляция данных	готово
Инструкция по запуску и наполнению баз	README: docker compose up + make seed	в работе
Запись всех сценариев	Скринкаст TUI: регистрация, чаты, реалтайм, поиск, роли, rate limit	план
Презентация для защиты	Диаграммы разделов 3-5 + путь сообщения + трейд-оффы	план
Описание вклада участников	Раздел 12 + история коммитов	шаблон
SQL: JOIN, GROUP BY, подзапросы	Раздел 9	готово
Аудит действий пользователей	Таблица audit_log + перечень событий (раздел 8)	готово

12. План работ и распределение вклада

Неделя	Задачи	Результат
1	Каркас монорепо, docker-compose (PG + Redis), proto-контракт, миграции, Register/Login/JWT, interceptor	Авторизация работает end-to-end
2	Чаты и участники, SendMessage, история (keyset), Subscribe-стрим (напрямую, без брокера)	Два клиента переписываются в реальном времени
3	Outbox + relay + Redis Streams + fanout, поиск (tsvector), роли, rate limit, read-курсор и «✓✓», TUI-полировка	MVP + расширения полностью
4	Артефакты: отчёт, запись сценариев, презентация, seed-скрипт, README, вклад участников	Пакет к защите

Распределение вклада (шаблон — заполнить по факту): участник А — Auth, модель авторизации, interceptor'ы; участник В — Chats/Messages, storage, миграции; участник С — брокер (relay/fanout), TUI, docker-compose и инструкция запуска. Отчёт и презентация — совместно.

13. Выводы

Спроектирован мессенджер, закрывающий все требования MVP и четыре расширения (UI, статусы прочтения, поиск, rate limit). Ключевые архитектурные решения — transactional outbox как единственный механизм надёжности, событийная модель «прочтение и typing — те же события, что и сообщения», read-курсор вместо статуса на каждое сообщение, keyset-пагинация и полнотекстовый поиск средствами PostgreSQL. Система умышленно проста в эксплуатации (один процесс, docker compose up) и при этом имеет понятный путь масштабирования: вынесение fanout в отдельный процесс поверх существующей consumer group.

14. Источники

- Документация PostgreSQL — postgresql.org/docs (полнотекстовый поиск, индексы, LISTEN/NOTIFY)
- Документация gRPC — grpc.io/docs (streaming, interceptors)
- Документация Redis Streams — redis.io/docs (XADD, XREADGROUP, consumer groups)
- Bubble Tea / Bubbles / Lipgloss — github.com/charmbracelet
- Паттерн Transactional Outbox — microservices.io/patterns/data/transactional-outbox.html
- Материалы вводной встречи и воркшопа кейса №1, VK Education